Trabajo práctico integrador: Mi Biblioteca Personal

1. Introducción

El objetivo de este trabajo práctico es desarrollar una aplicación web denominada Mi Biblioteca Personal, que permita a cada usuario administrar su propia colección de libros.

La aplicación deberá permitir registrar usuarios, iniciar y cerrar sesión, agregar libros a una biblioteca personal, modificar sus datos, eliminarlos, clasificarlos por género, registrar su estado de lectura y asignarles una puntuación.

Cada usuario deberá contar con un dashboard desde el cual pueda consultar información resumida sobre su biblioteca, conocer cuántos libros leyó, cuántos tiene pendientes y cuáles son sus géneros favoritos.

Además, deberá poder consultar estadísticas, buscar libros, filtrar la colección y exportar sus datos a un archivo CSV.

El sistema deberá desarrollarse aplicando programación orientada a objetos en PHP, arquitectura MVC, DAO, DTO, traits, manejo de errores y excepciones, logging, operaciones con archivos y persistencia en MySQL mediante PDO.

No se evaluará únicamente que las funcionalidades estén implementadas. También se tendrá en cuenta la organización del código, la separación de responsabilidades, la seguridad y la capacidad de justificar las decisiones de diseño.

2. Registro de usuarios

La aplicación deberá contar con un formulario de registro que permita crear una cuenta.

El formulario deberá solicitar:

- Nombre y apellido.

- Nombre de usuario.

- Correo electrónico.

- Contraseña.

- Confirmación de contraseña.

El sistema deberá validar los datos recibidos en el servidor, verificar que el nombre de usuario y el correo electrónico no estén registrados y controlar que las contraseñas coincidan.

Las contraseñas deberán almacenarse utilizando "password_hash()" y verificarse posteriormente con "password_verify()". No se permitirá guardar contraseñas en texto plano.

Los datos de los usuarios deberán persistirse en MySQL mediante PDO.

Si se produce un error durante el registro, el sistema deberá informar al usuario sin mostrar consultas SQL, rutas internas ni detalles técnicos sensibles.

3. Inicio y cierre de sesión

Los usuarios registrados deberán poder iniciar sesión mediante su nombre de usuario o correo electrónico y su contraseña.

La aplicación deberá:

- Verificar las credenciales ingresadas.
- Crear una sesión autenticada.
- Regenerar el identificador de sesión después de iniciar sesión.
- Impedir el acceso a las funcionalidades privadas sin autenticación.
- Permitir cerrar sesión de forma segura.
- Evitar que un usuario pueda consultar, modificar o eliminar libros pertenecientes a otro usuario.

Los intentos fallidos de inicio de sesión deberán registrarse en el sistema de logging, sin guardar las contraseñas ingresadas.

4. Gestión de libros

Cada usuario podrá administrar su propia colección.

La aplicación deberá permitir:

Agregar libros. El usuario podrá registrar un libro mediante un formulario con los siguientes datos:

- Título.
- Autor.
- Género literario.
- Año de publicación.
- Editorial.
- Estado de lectura.

- Puntuación personal.

- Reseña u observaciones.

Consultar libros. Se deberá mostrar un listado de los libros registrados, con paginación y acceso al detalle de cada uno.

Modificar libros. El usuario podrá actualizar los datos de sus libros.

Eliminar libros. El sistema deberá solicitar confirmación antes de eliminarlos y verificar que pertenezcan al usuario autenticado.

Buscar y filtrar. Se deberá permitir buscar por título o autor y filtrar por género, estado de lectura y puntuación.

Los datos deberán validarse en el servidor antes de persistirse. Las consultas SQL deberán realizarse mediante PDO y sentencias preparadas.

El sistema deberá contemplar los siguientes estados de lectura:

- Pendiente.

- Leyendo.

- Leído.

- Abandonado.

La puntuación será opcional y deberá admitir valores entre 1 y 5. Solo los libros con estado «Leído» podrán recibir una puntuación.

5. Gestión de géneros literarios

La aplicación deberá permitir clasificar los libros por género.

Cada género tendrá un nombre y una descripción opcional. La aplicación deberá incluir algunos géneros iniciales, como novela, cuento, poesía, ensayo, ciencia ficción, fantasía, terror y filosofía.

Los géneros deberán almacenarse en MySQL y relacionarse con los libros mediante claves foráneas.

Para simplificar el alcance, los géneros serán compartidos por todos los usuarios y su administración podrá quedar a cargo del sistema, sin necesidad de desarrollar un panel administrativo completo.

6. Dashboard personal

Después de iniciar sesión, cada usuario accederá a un dashboard con información resumida de su biblioteca.

Como mínimo, deberá mostrar:

- Cantidad total de libros registrados.
- Cantidad de libros leídos.
- Cantidad de libros pendientes.
- Cantidad de libros actualmente en lectura.
- Promedio de las puntuaciones asignadas.
- Géneros más frecuentes.
- Últimos libros agregados.
- Últimos libros marcados como leídos.

Los datos deberán obtenerse a partir de consultas a MySQL. No se permitirá calcular todas las estadísticas recorriendo indiscriminadamente la colección completa en PHP cuando puedan resolverse mediante consultas SQL apropiadas.

El dashboard deberá incluir enlaces a las funcionalidades principales: agregar un libro, consultar la colección, buscar libros y exportar la biblioteca.

7. Historial de actividad

El sistema deberá registrar determinadas acciones realizadas por el usuario.

Como mínimo, se deberá registrar:

- Alta de un libro.
- Modificación de un libro.
- Eliminación de un libro.
- Cambio de estado de lectura.
- Asignación o modificación de una puntuación.
- Exportación de la biblioteca.

Cada registro deberá incluir el usuario que realizó la acción, el tipo de operación, una descripción breve y la fecha y hora.

El historial deberá almacenarse en MySQL y podrá consultarse desde una sección de la aplicación.

El usuario únicamente podrá consultar su propio historial.

8. Manejo de archivos

La aplicación deberá incorporar operaciones con archivos del servidor.

Se deberán implementar las siguientes funcionalidades:

A. Exportación de la biblioteca

El usuario podrá exportar su colección a un archivo CSV que contenga los datos principales de sus libros.

El archivo deberá generarse a partir de la información almacenada en MySQL y guardarse en un directorio específico de la aplicación.

El sistema deberá controlar errores al crear directorios, abrir archivos, escribir información y cerrar recursos.

Los archivos exportados no deberán quedar disponibles para cualquier visitante mediante una URL pública. La descarga deberá pasar por un controlador que verifique la sesión y la autorización correspondiente.

B. Registro de logs

El sistema deberá escribir registros de actividad técnica en archivos de texto. Estos archivos deberán almacenarse separados de las exportaciones realizadas por los usuarios.

C. Gestión de errores de archivos

Se deberán contemplar los errores que puedan producirse por falta de permisos, directorios inexistentes, fallos de escritura o recursos no disponibles.

Las operaciones deberán informar los problemas mediante el sistema centralizado de logging y mostrar mensajes apropiados al usuario.

9. Programación orientada a objetos en PHP

La aplicación deberá desarrollarse utilizando clases y objetos.

Se deberán identificar las entidades principales y las responsabilidades de cada clase. Como mínimo, el diseño deberá contemplar clases relacionadas con:

- Usuarios.
- Libros.
- Géneros.
- Historial de actividad.
- Autenticación.
- Gestión de la biblioteca.
- Acceso a la base de datos.
- Logging.

- Manejo de errores.

- Exportación de archivos.

Los atributos deberán tener una visibilidad apropiada y las clases deberán exponer métodos que representen sus responsabilidades.

No se permitirá concentrar toda la lógica de la aplicación en un único archivo PHP ni utilizar clases que simplemente agrupen código sin una responsabilidad clara.

10. Traits

Se deberá implementar al menos un trait que permita reutilizar un comportamiento transversal.

Por ejemplo, se podrá implementar un "LoggableTrait" para facilitar el registro de eventos desde determinados servicios.

El trait deberá utilizar una dependencia de logging correctamente definida, sin crear conexiones a la base de datos ni encargarse de la persistencia de los libros.

El estudiante deberá justificar por qué el uso de un trait resulta apropiado en ese caso y explicar sus diferencias con la herencia y la composición.

No se considerará suficiente implementar un trait sin una finalidad concreta o utilizarlo solamente para cumplir formalmente con la consigna.

11. DTO: objetos de transferencia de datos

Se deberán implementar DTO para transportar datos entre las distintas capas de la aplicación.

Como mínimo, se deberá utilizar un DTO para el registro de usuarios y otro para las operaciones de creación o modificación de libros.

Por ejemplo:

- "UsuarioRegistroDTO".

- "LibroDTO".

- "FiltroLibrosDTO".

Los DTO deberán definir claramente qué datos transportan y no deberán encargarse de ejecutar consultas SQL ni de generar HTML.

El estudiante deberá explicar qué ventajas aporta utilizar DTO frente a pasar arrays asociativos entre las capas y en qué situaciones conviene validar los datos al construirlos.

12. DAO: objetos de acceso a datos

Se deberán implementar DAO para encapsular las operaciones de persistencia.

Como mínimo, la aplicación deberá contar con objetos de acceso a datos para:

- Usuarios.
- Libros.
- Géneros.
- Historial de actividad.

Los DAO serán responsables de ejecutar las consultas SQL mediante PDO y de transformar los resultados en estructuras de datos u objetos definidos por el diseño.

Deberán contemplar las operaciones necesarias para crear, consultar, modificar y eliminar registros, así como las búsquedas y consultas de estadísticas que correspondan.

La lógica de negocio, como decidir qué puntuación puede recibir un libro o qué usuario está autorizado para modificarlo, no deberá delegarse exclusivamente en los DAO.

13. Arquitectura MVC

La aplicación deberá organizarse utilizando el patrón Modelo-Vista-Controlador.

Modelo y servicios: representarán los datos y concentrarán las reglas del negocio.

Vista: se encargará de presentar los formularios, listados, estadísticas y mensajes al usuario.

Controlador: recibirá las solicitudes, verificará los datos de entrada, coordinará las operaciones correspondientes y seleccionará la respuesta o vista adecuada.

La arquitectura deberá mantener separadas las responsabilidades de presentación, lógica de negocio y acceso a datos.

No se permitirá ejecutar consultas SQL directamente desde las vistas ni incluir toda la lógica de negocio dentro de los controladores.

El estudiante deberá explicar el recorrido de una solicitud desde el navegador hasta la base de datos y el regreso de la respuesta al usuario.

14. Manejo de errores, excepciones y logging

La aplicación deberá implementar un sistema centralizado para gestionar errores y registrar información útil para el diagnóstico.

Se deberá configurar un manejador de errores de PHP y definir una política de tratamiento de excepciones.

El sistema deberá distinguir entre errores de validación, errores esperables del negocio y fallos inesperados de infraestructura.

Se deberá utilizar "try/catch" en las operaciones que puedan lanzar excepciones y en los límites apropiados de cada capa.

Como mínimo, se deberán contemplar:

- Errores de conexión con MySQL.
- Excepciones de PDO.
- Fallos durante la creación o modificación de libros.
- Errores de lectura y escritura de archivos.
- Intentos fallidos de autenticación.

- Errores durante la exportación de datos.

Se deberá implementar un logger que permita registrar niveles de severidad, como "INFO", "WARNING" y "ERROR".

Cada registro deberá incluir fecha y hora, nivel, mensaje y contexto suficiente para investigar el problema.

En producción, los errores no deberán mostrarse directamente al usuario. Los detalles técnicos deberán quedar en los logs, sin registrar contraseñas ni otros datos sensibles.

El estudiante deberá explicar la diferencia entre el manejo global de errores, las excepciones capturadas localmente y los registros generados por el logger.

15. Persistencia en MySQL mediante PDO

Toda la información principal de la aplicación deberá persistirse en una base de datos MySQL.

Se deberá implementar una clase responsable de crear y administrar la conexión PDO, configurando adecuadamente el manejo de excepciones y el juego de caracteres.

Las consultas deberán utilizar sentencias preparadas y parámetros enlazados. Se deberá evitar la concatenación de datos ingresados por el usuario dentro de las consultas SQL.

El diseño de la base de datos deberá contemplar claves primarias, claves foráneas, restricciones de unicidad e índices apropiados.

Las operaciones que requieran consistencia entre varias escrituras deberán utilizar transacciones cuando corresponda.

Se deberá entregar un script SQL para crear las tablas e insertar los géneros iniciales.

La aplicación no deberá depender de datos precargados manualmente desde una herramienta externa para poder funcionar.

16. Seguridad mínima obligatoria

La aplicación deberá contemplar, como mínimo:

- Contraseñas almacenadas mediante hashes seguros.
- Consultas preparadas con PDO.
- Validación de datos en el servidor.
- Escape apropiado de los datos al generar HTML.
- Protección contra CSRF en operaciones que modifican información.
- Control de autenticación y autorización.
- Protección de los libros y del historial de cada usuario.
- Gestión segura de sesiones.
- Manejo de errores sin revelar información sensible.
- Protección de las exportaciones y de los archivos de logs.

La validación del lado del cliente podrá mejorar la experiencia de uso, pero nunca reemplazará las validaciones del servidor.

17. Entregables

Cada estudiante o grupo deberá presentar:

1. Código fuente completo y organizado.
2. Script SQL de creación de la base de datos y carga inicial.
3. Archivo de configuración de ejemplo, sin credenciales reales.
4. Instrucciones de instalación y ejecución.
5. Aplicación funcional con registro, login, dashboard y gestión de libros.
6. Exportación de la biblioteca a CSV.
7. Sistema de logging y manejo de errores.
8. Explicación de la arquitectura MVC, los DAO, los DTO y los traits utilizados.

9. Pruebas de las funcionalidades principales y de los casos de error.

18. Criterios de evaluación

Se evaluará el funcionamiento de la aplicación, la calidad del diseño y la correcta aplicación de los contenidos de la materia.

El código deberá ser legible, consistente y mantenible. Se tendrá en cuenta la separación de responsabilidades, el tratamiento de errores, la seguridad, la persistencia y la justificación de las decisiones técnicas.

No será suficiente que la aplicación funcione en el caso ideal: también deberá comportarse correctamente frente a datos inválidos, registros inexistentes, accesos no autorizados y fallos de infraestructura.